SIGNAL + ADAPT

Exact Hardware Map & Complete Firmware

Pin-by-Pin Wiring + Full Arduino C++ Source

Single-channel EMG, no EXG Pill, no Servo Claw

BOARD:	Arduino Uno-class
FRONT END:	Muscle BioAmp Shield v0.3 (1 channel, A0)
ELECTRODES:	1x Muscle BioAmp Band + gel electrodes
ACTUATOR:	SG90-class hobby servo
WIRELESS:	ESP32 (telemetry/logging only, outside control loop)
FIRMWARE:	Single .ino — SIGNAL burst classifier + ADAPT remap table
DATE:	21 June 2026

1. Exact Bill of Materials

Every line below is either confirmed owned or a cheap, named, buyable part. Nothing here requires a BioAmp EXG Pill or a Servo Claw.

Component	Qty	Status	Role
Muscle BioAmp Shield v0.3	1	OWNED	Sole EMG front end, A0 only
Arduino Uno-class board	1	OWNED	Runs all control logic
Muscle BioAmp Band	1	OWNED	Electrode band, single site
Gel electrodes	as needed	OWNED	Skin contact
BioAmp Cable	1	OWNED	Band to Shield
STEMMA cables (x6)	0 used in core build	OWNED	Spare; not required for this build
ESP32 dev board	1	OWNED	Telemetry bridge only, outside control loop
SG90-class hobby servo	1	BUY, ~Rs 80-150	Switch-press lever actuator
9V battery + snap connector / 5V power bank	1	BUY, ~Rs 50-250	Isolated power
Coin vibration motor + NPN transistor + flyback diode	1 set	BUY, optional, ~Rs 40-60	Haptic confirmation
3.5mm switch jack + plug, or small relay	1 set	BUY, optional, ~Rs 50-100	Electrical (non-mechanical) switch integration variant
Micro-SD module or onboard flash	1	BUY, optional, ~Rs 100-150	Local session logging for ADAPT, if not logging via ESP32/host

2. Pin Map (Complete)

Pin	Function	Notes
A0	sEMG signal in	Muscle BioAmp Shield onboard AFE output. The only biopotential channel.
D9	Servo PWM out	Shield's dedicated hobby-servo header. SG90 signal wire.
D6	Haptic motor PWM out	Through NPN transistor base (1k resistor) + flyback diode across motor. Never direct from pin.
D2 (optional)	Relay/reed switch drive	Only if using the electrical 3.5mm-jack switch variant instead of the mechanical lever.
TX0/RX0 (Serial0)	USB serial to host PC	Used for calibration commands and telemetry during development.
TX1/RX1 (SoftwareSerial, if board lacks 2nd hardware UART)	Serial to ESP32	Telemetry only; read-only from ESP32's perspective, never blocks the Arduino loop.
VIN	Battery power in	9V snap connector. Do not power from USB during real use.
5V / GND	Shield + servo + motor power rail	Shield supplies regulated 5V to servo directly.

3. Step-by-Step Wiring

Step 1: Shield on Arduino

Seat the Muscle BioAmp Shield v0.3 firmly on the Arduino's headers. This is the only biopotential front end — A0 carries the EMG signal, and the Shield's onboard servo header becomes available.

Step 2: Electrode site

Apply gel electrodes per the Band's standard layout on the chosen muscle site (cheek, eyebrow, forearm flexor, or whichever site is most reliably controllable for the user).

Step 3: Band to Shield

Connect the Muscle BioAmp Band to the Shield's onboard EMG input using the BioAmp Cable. Signal now present on A0.

Step 4: Servo to Shield

Plug the SG90-class servo into the Shield's dedicated hobby servo header (3-pin: Signal/5V/GND). Signal wire lands on D9. The Shield supplies regulated 5V directly to the servo — no separate BEC needed.

Step 5: Lever or switch-press mechanism

Attach a simple lever arm (3D-printed, or hot-glue-and-cardboard for a prototype) to the servo horn, positioned to physically depress the target device's existing switch button or jack. No electrical integration with the target device required for this variant.

Step 6 (optional): Vibration motor to D6

Drive through an NPN transistor (base via ~1k resistor from D6) with a flyback diode across the motor leads. Never connect a motor directly to a GPIO pin.

Step 7 (optional): Relay/reed switch to D2

Only if using the electrical 3.5mm-jack variant instead of the mechanical lever — closes the target device's switch circuit directly. Skip if using Step 5's lever.

Step 8: ESP32 telemetry link

Connect Arduino's spare serial TX to ESP32's RX (or use a second USB/serial adapter). This is read-only telemetry; the ESP32 cannot send control commands back into the real-time loop.

Step 9: Power

Connect 9V battery via snap connector to VIN, or a 5V power bank to the 5V rail. Use battery power during any real signal acquisition — USB power from a laptop introduces 50Hz switching noise onto the analog ground plane.

4. Electrode Placement

Site	Electrode Layout	Best For
Cheek (masseter)	Two electrodes ~2cm apart over the muscle belly, reference on a nearby bony point (e.g. mastoid or zygomatic arch)	Good for users with reliable jaw clench but limited limb control
Eyebrow (frontalis/corrugator)	Two electrodes along the brow, reference on the forehead midline or mastoid	Good for users who can reliably raise or furrow a brow
Forearm flexor	Two electrodes ~2cm apart over the flexor digitorum belly, reference on the wrist or elbow	Good for users with residual finger/wrist flexor control

Always prep skin before placing gel electrodes — clean with an alcohol wipe if available, let dry, then apply. Re-seat electrodes if the signal looks noisy or flat on first power-on.

5. Power Rules

- **Battery only during real use.** USB power is fine for development/flashing firmware on a bench, but switch to 9V battery or power bank before doing any real signal acquisition or fitting on a user — laptop switching supplies inject noise directly onto the EMG ground plane.
- **Common star ground.** Route all grounds (Shield, servo, haptic motor, relay if used) to a single common ground point rather than daisy-chaining, to avoid servo current spikes corrupting the microvolt-scale EMG signal.
- **Never drive the haptic motor or relay directly off a GPIO pin.** Always through a transistor with a flyback diode across the inductive load.
- **The ESP32, if powered separately, still needs a common ground with the Arduino** for the serial telemetry link to read cleanly.

6. Complete Firmware — Full Listing

One sketch, SIGNAL_ADAPT.ino. Combines the original SIGNAL burst classifier with ADAPT's remappable command table and serial-driven remap parser. Paste sections in order into one file. Requires only the bundled Servo library — nothing else.

6.1 Header, Pins, Constants

```
/* ================================================================
 SIGNAL + ADAPT -- single-channel EMG switch-access controller
 with remappable command vocabulary
 Hardware : Muscle BioAmp Shield v0.3 (A0 only) + Arduino
 Actuator : SG90-class servo (D9) -> mechanical switch press
 Optional : vibration motor (D6), relay (D2), ESP32 telemetry
 ================================================================ */
#include <Servo.h>

enum DriveMode { SERVO_MODE, RELAY_MODE };
DriveMode driveMode = SERVO_MODE;
const uint8_t PIN_EMG=A0, PIN_SERVO=9, PIN_HAPTIC=6, PIN_RELAY=2;
const uint16_t FS_HZ = 1000;
const uint32_t SAMPLE_US = 1000000UL / FS_HZ;
uint32_t lastSampleUs = 0;

Servo lever; int LEVER_REST=0, LEVER_PRESS=45;
```

6.2 Signal State

```
float emgDC=512, emgMS=0, emgRMS=0;
int tb0=0, tb1=0, tb2=0; float tkeoEnv=0;

struct Params {
  float emgBase=0.02f;
  float tkOnset=8000.0f;
  uint16_t debounceMs=120;
  uint16_t shortMaxMs=250;
  uint16_t longMinMs=600;
  uint16_t doubleWinMs=600;
  uint16_t pressMs=300;
  uint8_t  spasmCount=5;
  uint16_t spasmWinMs=2000;
} P;
```

6.3 Burst FSM State

```
enum State : uint8_t { S_IDLE, S_BURST, S_CLASSIFY, S_ACTUATE, S_CONFIRM, S_LOCKOUT };
State state = S_IDLE;
uint32_t tBurstStart=0, tBurstEnd=0;
uint8_t burstCount=0;
uint32_t burstTimes[4];
enum Cmd : uint8_t { CMD_NONE, CMD_SHORT, CMD_LONG, CMD_DOUBLE };
Cmd lastCmd = CMD_NONE;

// session counters (for telemetry / ADAPT covariate logging on host)
uint16_t attemptCount[3] = {0,0,0};   // indexed by Cmd-1
uint16_t successCount[3] = {0,0,0};
uint16_t falseNegCount = 0;
```

6.4 ADAPT: Remappable Command Table

```
// --- ADAPT addition: logical commands + remappable pattern table ---
enum LogicalCmd : uint8_t { LCMD_SELECT, LCMD_BACK, LCMD_HOME, LCMD_NONE };

// a 'pattern' can be a single Cmd, or a 2-step compound (for migrated commands)
struct PatternMap {
  Cmd  primary;      // first required burst pattern
  Cmd  secondary;    // CMD_NONE if not a compound pattern
  LogicalCmd logical;
} activeMap[3] = {
```

```
  { CMD_SHORT,  CMD_NONE, LCMD_SELECT },
  { CMD_LONG,   CMD_NONE, LCMD_HOME   },
  { CMD_DOUBLE, CMD_NONE, LCMD_BACK   },
};

bool practiceMode = false;
uint8_t practiceSessionsLeft = 0;

// compound-pattern matcher state (for migrated commands like SHORT_SHORT)
Cmd pendingPrimary = CMD_NONE;
uint32_t pendingPrimaryTime = 0;
const uint16_t COMPOUND_WIN_MS = 700;
```

6.5 Core EMG Processing (unchanged from SIGNAL)

```
inline void updateEMG(int raw){
  emgDC += 0.0008f*(raw-emgDC);
  float x = raw - emgDC;
  emgMS += 0.01f*(x*x - emgMS);
  emgRMS = sqrtf(emgMS);
  tb2=tb1; tb1=tb0; tb0=(int)x;
  float psi = (float)tb1*tb1 - (float)tb0*tb2;
  if (psi<0) psi=0;
  tkeoEnv += 0.03f*(psi-tkeoEnv);
}
inline bool onsetFired(){ return tkeoEnv > P.tkOnset; }
inline bool belowFloor(){ return emgRMS < P.emgBase*1.5f; }
```

6.6 setup()

```
void setup(){
  Serial.begin(115200);
  lever.attach(PIN_SERVO); lever.write(LEVER_REST);
  pinMode(PIN_HAPTIC, OUTPUT); analogWrite(PIN_HAPTIC, 0);
  pinMode(PIN_RELAY, OUTPUT); digitalWrite(PIN_RELAY, LOW);
  Serial.println(F("SIGNAL+ADAPT ready. 'c'=calibrate."));
}
```

6.7 Burst FSM — IDLE through CLASSIFY

```
void fsmStep(){
  uint32_t now = millis();
  switch(state){
    case S_IDLE:
      if(onsetFired()){ tBurstStart=now; burstCount=1; state=S_BURST; }
      break;
    case S_BURST:
      if(belowFloor() && (now-tBurstStart)>20){
        tBurstEnd=now; state=S_CLASSIFY;
      }
      break;
    case S_CLASSIFY: {
      uint16_t dur = tBurstEnd - tBurstStart;
      if(now - tBurstEnd < P.doubleWinMs && onsetFired()){
        burstCount++; tBurstStart=now; state=S_BURST; break;
      }
      if(burstCount>=2){ lastCmd = CMD_DOUBLE; }
      else if(dur >= P.longMinMs){ lastCmd = CMD_LONG; }
      else if(dur <= P.shortMaxMs){ lastCmd = CMD_SHORT; }
      else { lastCmd = CMD_NONE; falseNegCount++; }
      for(int i=3;i>0;i--) burstTimes[i]=burstTimes[i-1];
      burstTimes[0]=now;
      if(now-burstTimes[3] < P.spasmWinMs && burstTimes[3]!=0){
        state=S_LOCKOUT; tBurstStart=now; break;
      }
      state = (lastCmd!=CMD_NONE) ? S_ACTUATE : S_IDLE;
      burstCount=0;
      break; }
```

6.8 ADAPT: Logical-Command Resolution (the new layer)

```
// resolves a detected burst pattern into a logical command, handling
// both simple (single-pattern) and compound (migrated, 2-pattern) maps
LogicalCmd resolveLogical(Cmd detected, uint32_t now){
  for(int i=0;i<3;i++){
    PatternMap &m = activeMap[i];
    if(m.secondary == CMD_NONE){
      // simple mapping
      if(m.primary == detected) return m.logical;
    } else {
      // compound mapping: needs primary THEN secondary within window
      if(pendingPrimary == CMD_NONE){
        if(m.primary == detected){
          pendingPrimary = detected; pendingPrimaryTime = now;
          return LCMD_NONE; // waiting for second half
        }
      } else if(now - pendingPrimaryTime < COMPOUND_WIN_MS){
        if(m.primary==pendingPrimary && m.secondary==detected){
          pendingPrimary = CMD_NONE;
          return m.logical;
        }
      } else {
        pendingPrimary = CMD_NONE; // window expired, reset
      }
    }
  }
  return LCMD_NONE;
}
```

6.9 Burst FSM — ACTUATE, CONFIRM, LOCKOUT

```
    case S_ACTUATE: {
      LogicalCmd lc = resolveLogical(lastCmd, now);
      if(lc != LCMD_NONE){
        if(lc == LCMD_BACK && driveMode == RELAY_MODE)
          { digitalWrite(PIN_RELAY, HIGH); }
        lever.write(LEVER_PRESS);
        if(now - tBurstEnd > P.pressMs){
          lever.write(LEVER_REST);
          digitalWrite(PIN_RELAY, LOW);
          state = S_CONFIRM; tBurstEnd = now;
        }
      } else {
        // pattern didn't complete a logical command yet (e.g. mid-compound)
        // or matched nothing -- return to idle without actuating
        state = S_IDLE;
      }
      break; }
    case S_CONFIRM:
      analogWrite(PIN_HAPTIC, 180);
      if(now - tBurstEnd > 80){ analogWrite(PIN_HAPTIC, 0); state = S_IDLE; }
      break;
    case S_LOCKOUT:
      if(now - tBurstStart > 2000){ state = S_IDLE; }
      break;
  }
}
```

6.10 ADAPT: Serial Remap Parser

```
LogicalCmd parseLogical(String t){
  if(t=="SELECT") return LCMD_SELECT;
  if(t=="HOME")   return LCMD_HOME;
  if(t=="BACK")   return LCMD_BACK;
  return LCMD_NONE;
}
Cmd parseCmdName(String t){
  if(t=="SHORT")  return CMD_SHORT;
  if(t=="LONG")   return CMD_LONG;
  if(t=="DOUBLE") return CMD_DOUBLE;
  return CMD_NONE;
}

// format from host: REMAP,<flagged_logical>,<PATTERN1>[_<PATTERN2>]
// e.g.  REMAP,HOME,SHORT_SHORT
```

```
void parseRemapCommand(String line){
  int c1 = line.indexOf(',');
  int c2 = line.indexOf(',', c1+1);
  String flagged = line.substring(c1+1, c2);
  String newPat  = line.substring(c2+1);
  LogicalCmd target = parseLogical(flagged);
  int us = newPat.indexOf('_');
  Cmd p1, p2;
  if(us == -1){ p1 = parseCmdName(newPat); p2 = CMD_NONE; }
  else {
    p1 = parseCmdName(newPat.substring(0,us));
    p2 = parseCmdName(newPat.substring(us+1));
  }
  for(int i=0;i<3;i++){
    if(activeMap[i].logical == target){
      activeMap[i].primary = p1;
      activeMap[i].secondary = p2;
    }
  }
  practiceMode = true; practiceSessionsLeft = 5;
  Serial.print(F("REMAPPED ")); Serial.println(flagged);
}
```

6.11 Calibration

```
float measureRMS(uint16_t ms){
  uint32_t t0=millis(); float acc=0; uint32_t n=0;
  while(millis()-t0<ms){ updateEMG(analogRead(PIN_EMG)); acc+=emgRMS; n++; delay(1); }
  return acc/max((uint32_t)1,n);
}
void runCalibration(){
  Serial.println(F("CAL relax 3s")); delay(800); P.emgBase=measureRMS(3000);
  Serial.println(F("CAL twitch x3")); delay(800);
  P.tkOnset = tkeoEnv*0.6f + 3000.0f;
  Serial.println(F("CAL done."));
}
```

6.12 Main Loop — Sampling, Telemetry, Serial Commands

```
void loop(){
  if(Serial.available()){
    String line = Serial.readStringUntil('\n');
    if(line == "c") runCalibration();
    else if(line.startsWith("REMAP,")) parseRemapCommand(line);
  }
  uint32_t nowUs = micros();
  if(nowUs - lastSampleUs < SAMPLE_US) return;
  lastSampleUs = nowUs;
  updateEMG(analogRead(PIN_EMG));
  fsmStep();
  static uint8_t dec=0;
  if(++dec>=10){ dec=0;
    // telemetry row consumed by host for ADAPT covariate logging:
    // t_ms, emgRMS, tkeoEnv, state, lastCmd, falseNegCount
    Serial.print(millis()); Serial.print(',');
    Serial.print(emgRMS,1); Serial.print(',');
    Serial.print(tkeoEnv,0); Serial.print(',');
    Serial.print(state); Serial.print(',');
    Serial.print(lastCmd); Serial.print(',');
    Serial.println(falseNegCount);
  }
}
```

7. Firmware Section Index

Section	Content	Status
6.1	Header, pins, constants	Setup constants only, no logic
6.2	Signal state variables	EMG processing state, ADAPT tuning parameters
6.3	Burst FSM state	SIGNAL's original state machine variables, plus per-session counters for ADAPT logging
6.4	ADAPT remappable table	NEW — the activeMap structure that makes commands remappable
6.5	Core EMG processing	Unchanged from original SIGNAL — DC-block, RMS, TKEO
6.6	setup()	Pin init, servo attach, relay init
6.7	Burst FSM IDLE-CLASSIFY	Unchanged burst detection and pattern classification
6.8	ADAPT logical resolution	NEW — translates a detected burst pattern into a logical command, handling both simple and migrated compound patterns
6.9	Burst FSM ACTUATE-LOCKOUT	Modified to call resolveLogical() before actuating, and to support the optional relay path
6.10	ADAPT remap parser	NEW — parses REMAP serial commands from the host and updates activeMap
6.11	Calibration	Unchanged from SIGNAL
6.12	Main loop	Modified to also check for incoming REMAP commands each iteration, non-blocking

8. ESP32 Companion Sketch

Read-only telemetry relay. Cannot send anything into the Arduino's control loop — it only forwards what it receives over serial to a simple web page, and separately can log rows to local flash for the ADAPT covariate pipeline to consume later.

```
// esp32_bridge.ino -- read-only telemetry relay + optional local logging
#include <WiFi.h>
#include <WebServer.h>
#include <SPIFFS.h>

WebServer server(80);
String lastLine = "";
File logFile;

void handleRoot(){ server.send(200, "text/plain", lastLine); }

void setup(){
  Serial.begin(115200);          // from Arduino TX
  WiFi.begin("yourSSID","yourPASS");
  while(WiFi.status()!=WL_CONNECTED) delay(300);
  SPIFFS.begin(true);
  logFile = SPIFFS.open("/session_log.csv", FILE_APPEND);
  server.on("/", handleRoot);
  server.begin();
}

void loop(){
  if(Serial.available()){
    lastLine = Serial.readStringUntil('\n');
    if(logFile) logFile.println(lastLine);  // append for ADAPT host pickup
  }
  server.handleClient();
}
```

9. Calibration Command Reference

Serial Command	Effect
c	Run full calibration: relax baseline, then twitch x3 to set onset threshold
REMAP,,	Force a remap manually for testing, e.g. REMAP,HOME,SHORT_SHORT
(any other line)	Ignored

Tuning guide carried over from SIGNAL: if the system pre-positions on stray twitches, raise tkOnset; if SHORT and DOUBLE are confused, widen the gap between shortMaxMs and doubleWinMs; if LONG never triggers reliably, lower longMinMs or consider migrating it via ADAPT rather than fighting the threshold.

10. Troubleshooting

Symptom	Likely Fix
EMG looks like noise / 50Hz hum	On USB power -> switch to battery; re-check reference electrode; re-prep skin
Burst never classifies	Check A0 wiring to Shield; verify tkOnset isn't set too high from a bad calibration
Servo doesn't move on ACTUATE	Check D9 connection to Shield's SRV header; confirm 5V rail is live
REMAP command has no effect	Confirm exact format REMAP,, with no spaces; LOGICAL must be SELECT/HOME/BACK exactly
Compound pattern never completes	COMPOUND_WIN_MS (700ms) may be too short for the user's natural inter-burst timing -- increase it
ESP32 shows blank/stale telemetry	Check shared ground between Arduino and ESP32; verify baud rate matches on both ends (115200)
Haptic motor weak or pin runs hot	Confirm it's wired through the NPN transistor, not directly off D6

END OF HARDWARE & FIRMWARE REFERENCE

Generated 21 June 2026, 11:18 | Mapped strictly to owned hardware